Kluczowe aspekty przetwarzania transakcyjnego
Key aspects of transactional processing

dr hab. inż. Maciej Grzenda
M.Grzenda@mini.pw.edu.pl
http://www.mini.pw.edu.pl/~grzendam

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.

Objectives

- Transactional processing should guarantee ACID features including transaction isolation
- However, complete isolation of transactions would be against performance
- Hence, a balance between performance and isolation has to be sought
- Anomalies of transactional processing such as dirty reads may occur due to limited isolation
- Thus, in-depth understanding of the consequences of paralel transaction processing is of primary importance

Transaction isolation - problems

Account 456 - balance

1000 USD

1500 USD

1400 USD

Transaction A

```
BEGIN TRANSACTION
UPDATE Accounts set
   balance=balance+500
   where
   account_id=456
```

. . .

```
ROLLBACK
```

Transaction B

```
BEGIN TRANSACTION
UPDATE Accounts set
   balance=balance-100
   where account_id=456
```

```
UPDATE Accounts set
   balance=balance+100
   where account_id=8899
COMMIT
```

The first transaction could affect the result of the second one causing wrong ultimate account balance (1400 USD instead of 900 USD)

Transaction anomalies

- In reality, multiple transactions submitted by different applications from different workstations are executed in parallel by a DBMS
- Transaction isolation would be easily achieved if transactions were NOT executed in parallel i.e. serialised. However that would diminish DBMS performance
- Hence, transactions are executed in parallel and anomalies may occur

Transaction anomalies

Category	Description
Dirty reads *Brudne odczyty*	Occurs when a transaction reads data that has not yet been committed. If another transaction finally rolls back than the first transaction can read the data that has never really existed.
Lost updates *Utracone aktualizacje*	Occur when more than one transaction read the same data. The modifications made later by the first transaction can be overwritten by another transaction unaware of previously made changes. Thus the changes made first can be ignored and/or replaced with the original content.
Non-repeatable reads *Niepowtarzalne odczyty*	occurs when a transaction reads the same row twice but gets different data each time. For example, suppose transaction 1 reads a row. Transaction 2 updates or deletes that row and commits the update or delete. If transaction 1 rereads the row, it retrieves different row values or discovers that the row has been deleted
Phantoms *Rekordy typu fantom*	A phantom is a row that matches the search criteria but is not initially seen. For example, suppose transaction 1 reads a set of rows that satisfy some search criteria. Transaction 2 generates a new row that matches the search criteria for transaction 1. If transaction 1 re-executes the statement that reads the rows, it gets a different set of rows

Anomalies - solution

- Modern DBMSs make it possible to set transaction isolation level (*poziom izolacji transakcji*). Different levels are provided so as to achieve a balance between performance and required isolation
- Example:
  - Similarly to other DBMSs, Ms SQL Server provides a number of transaction isolation levels
  - SET TRANSACTION ISOLATION LEVEL LevelName

Isolation levels – Ms SQL Server

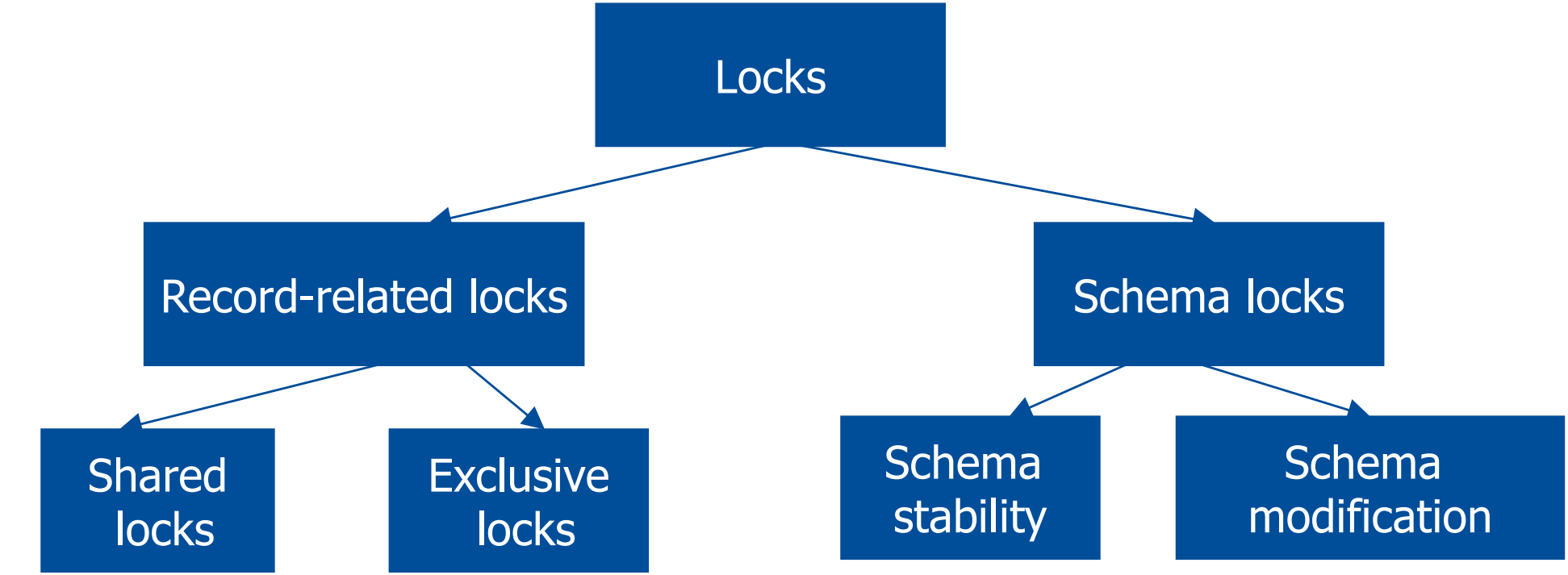
Isolation level	Dirty reads	Nonrepeatable reads	Phantoms	Remarks
READ UNCOMMITED	YES	YES	YES	The most limited synchronisation between transactions. A SELECT statement in transaction A may see data not committed yet by transaction B.
READ COMMITTED	NO	YES	YES	Only results of committed transactions can be seen. Still they can commit during execution of another transaction and cause nonrepeatable read and phantoms
REPEATABLE READ	NO	NO	YES	Phantom records can happen
SNAPSHOT	NO	NO	NO	A transaction will see the content of the database as it was at the start of the transaction
SERIALIZABLE	NO	NO	NO	Corresponds to serial execution of transactions, the data read by a transaction gets blocked

Locks
Blokady

- In order to satisfy requested transaction isolation level DBMS typically imposes locks. Locks can affect for instance:
  - record that is being modified,
  - range of records
  - the entire table including all its data and indexes
  - entire database
  - metadata
- Whilst processing an SQL statement DBMS checks whether resources (such as record, group of records, table or metadata) required for this statement to execute are locked. If so, the statement can be suspended

For details on locks and transactional processing in Ms SQL Server see https://docs.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?view=sql-server-ver15

Locks in Ms SQL Server: key categories



Update locks are imposed by DBMS in most cases automatically. They correspond to transaction isolation level and may allow read-only access to a modified resource (e.g. record).

Schema stability locks ensure that a db object (table, index etc.) will not be removed when used by another transaction. Schema modification locks prevent access to the db objects that are being modified

Ms SQL Server: the most important lock categories

Lock mode	Used when	Consequences
Shared	Used for read operations not changing data i.e. SQL SELECT	No other transactions can modify data during shared lock
Exclusive	Used for INSERT, UPDATE and DELETE to prevent multiple modifications to the same resource at the same time	No other transaction can modify locked data. Under default transaction isolation level, no other transaction can read data that is locked in exclusive mode.
Schema modification	Used during DDL statement on a table i.e. a statement modifying the table	Concurrent access to the table being modified is prevented
Schema stability	Applied when compiling and executing queries on the table.	Concurrent changes to the data are allowed, but changes to table structure i.e. DDL statements are prevented

Note that locks are applied and released automatically by a DBMS. Typically, they do not cause errors, but may suspend statements from other transactions.

Locks in practice

The balance of account 456

1000 USD

1500 USD

1000 USD

900 USD

Transaction A

```
BEGIN TRANSACTION
UPDATE Accounts set
   balance=balance+500
   where
   account_id=456
   . . .
ROLLBACK
```

(1) An exlusive lock is placed on record 456 by transaction A

Transaction B

```
BEGIN TRANSACTION
UPDATE Accounts set
   balance=balance-100
   where account_id=456
UPDATE Accounts set
   balance=balance+100
   where account_id=8899
COMMIT
```

(2) The update statement encounters a lock and is suspended

(3) ROLLBACK reverts all changes and releases all the locks including the exclusive lock

(4) The UPDATE statement is woken up and does its change

Concurrency issues – example 1

DB Session #1

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED

begin transaction

update customers set ordercount=2
where customerId='ALFKI'
```

```
ROLLBACK
```

```
COMMIT
```

DB Session #2

```
SET TRANSACTION ISOLATION
LEVEL READ UNCOMMITTED

begin transaction

Select * from customers
where customerId='ALFKI'
```

DB Session#2 is not suspended. Dirty reads are possible, as transaction #1 may be cancelled

Concurrency issues – example 2

DB Session #1

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED

begin transaction

update customers set ordercount=2
where customerId='ALFKI'
```

DB Session #2

```
SET TRANSACTION ISOLATION
LEVEL READ COMMITTED

begin transaction

Select * from customers
where customerId='ALFKI'
```

```
ROLLBACK
```

```
COMMIT
```

DB Session#2 is suspended until session#1 is committed or cancelled

Concurrency issues – example 3

DB Session #1

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED

begin transaction

select * from customers
where customerId='ALFKI'
```

DB Session #2

```
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED

begin transaction

update customers set ordercount=3
where customerId='ALFKI'
commit
```

```
select * from customers
where customerId='ALFKI'
```

DB Session#2 is not suspended. Non-repeatable reads are possible in session#1

Concurrency issues – example 4

DB Session #1

```
SET TRANSACTION ISOLATION LEVEL
REPEATABLE READ

begin transaction

select * from customers
where customerId='ALFKI'
```

DB Session #2

```
SET TRANSACTION ISOLATION
LEVEL READ COMMITTED

begin transaction

update customers set ordercount=3
where customerId='ALFKI'
commit
```

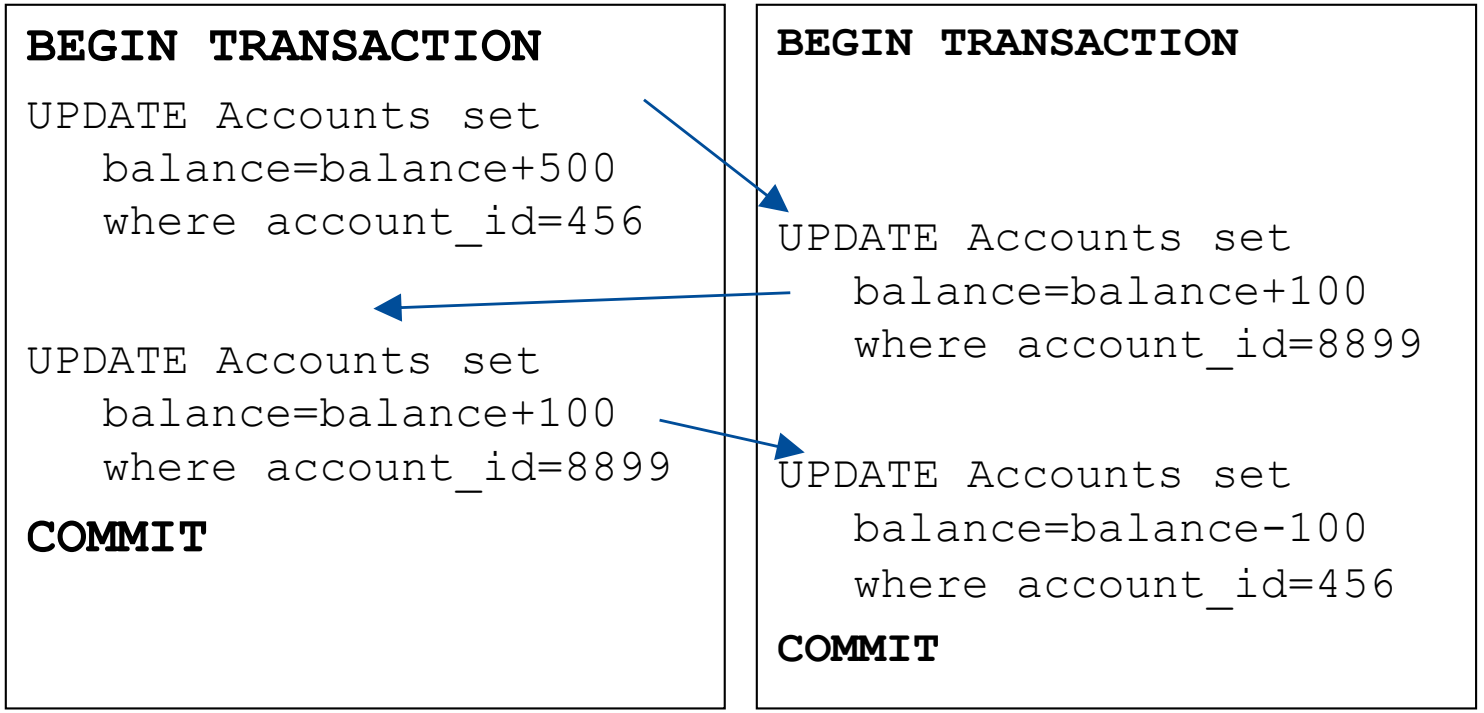
```
select * from customers
where customerId='ALFKI'
commit
```

DB Session#2 is suspended. Non-repeatable reads are not possible in session#1, as all the changes to its data are suspended.

Deadlocks
Zakleszczenia

- Deadlock – a combination of locks that prevents more than one transaction from execution and can not be resolved without terminating one of the transactions
- To terminate a transaction means to use ROLLBACK for the transaction
- This means cancelling all the updates and should be avoided

Deadlock – a simple example



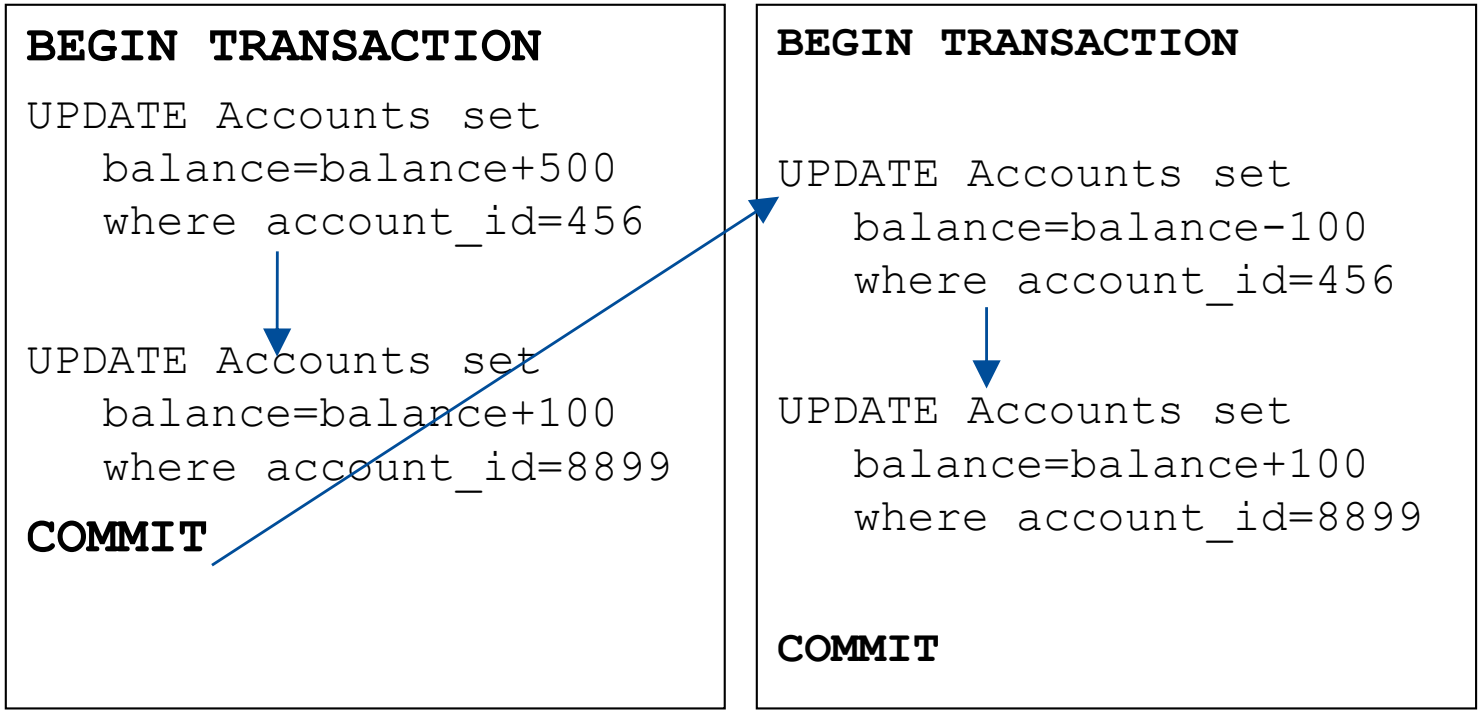
(1) Locks Account 456
(3) Waits for the lock on account 8899 to be released

(2) Locks Account 8899
(4) Waits for the lock on account 456 to be released - deadlock occurs

Deadlocks- how to avoid them?

- Always lock the resources in the same sequence i.e. issue statements locking resources (frequently UPDATE statements) in the same sequence
- Example:
  - Always lock account with lowest applicable number first
  - Always lock account table before customers table

Eliminating the risk of **this** deadlock



(1) Locks account 456
(2) Locks account 8899
(3) At COMMIT releases both locks

(1) Waits for the lock on account 456 to be released
(2) Locks account 456
(2) Locks account 8899
(3) At COMMIT releases both locks

Locks – how to monitor them?

- Ms SQL Server:
  - Enterprise Manager/SQL Server Management Studio can be used for that purpose (Current activity subtree)
  - Alternatively system stored procedure sp_lock can be started
- Note that locks are released when a transaction terminates (with COMMIT or ROLLBACK)
- Can locks influence system performance?

Save points

- Can be used to prevent rolling back entire transaction and releasing all locks acquired so far
- Are supposed to be used when unlikely errors may occur and we do not want to restart the entire complex processing then
- Example (Ms SQL Server):

```
BEGIN TRANSACTION A
   /* do some updates #1 */
   SAVE TRAN B
   /* do some updates #2 */

   ROLLBACK TRAN B
   /* do some updates #3 */
   COMMIT
```

Places a kind of label to be referred to in the future if necessary

Does NOT roll back the entire transaction – just the changes executed after SAVE TRAN B

Transactions – execution time

Category	Description
Standard transactions *transakcje*	• Impose locks on accessed resources. Therefore, not to decrease transaction throughput, a transaction must be possibly short. Typical transaction takes less than a second to complete. • Prevailing majority of transactions are short time transactions. • Hence, by transaction unless otherwise stated, a short-running transaction is meant
Long-running transactions *Długotrwałe transakcje*	• Imagine a modification of the design of large new investment (water supply network for new part of the city). It may take days to complete new version of the project. We would not like anyone else to see the changes before that time. • Long-running transaction can not be based on locks. That would stop any work on the project (e.g. read-only access) for a long time of possibly many days. Instead, under long-running transactions, a logical copy of modified data is created. • Not as popular as short-running transactions.

Any problems here?

/* Generate code that takes current balance of bank account from a relational database and then updates it by 100 */

```
public void updateAccountBalance(String accountId) {
String url = "jdbc:mysql://localhost:3306/bank";
String user = "root";
String password = "password";

String selectQuery = "SELECT Balance FROM Accounts WHERE AccountID = ?";
String updateQuery = "UPDATE Accounts SET Balance = ?  WHERE AccountID = ?";

try (Connection conn = DriverManager.getConnection(url, user, password);
PreparedStatement selectStmt = conn.prepareStatement(selectQuery);
PreparedStatement updateStmt = conn.prepareStatement(updateQuery)) {

// Retrieve current balance
selectStmt.setString(1, accountId);
try (ResultSet rs = selectStmt.executeQuery()) {
if (rs.next()) {
double currentBalance = rs.getDouble("Balance")+100;

// Update balance
updateStmt.setDouble(1, currentBalance);
updateStmt.setString(2, accountId);
int rowsUpdated = updateStmt.executeUpdate();
if (rowsUpdated > 0) {
System.out.println("Balance updated successfully.");
}
} else {
System.out.println("Account not found.");
}
}
} catch (SQLException e) {
e.printStackTrace();
}
}
```

Summary

- Complete isolation of transactions would mean avoiding concurrent execution of transactions
- This would largely decrease the performance of transactional processing and of database systems, in turn
- Instead, transactions are executed in parallel
- Hence, in-depth understanding of transaction isolation levels, locks, and possible anomalies is vital for anyone working with databases

Warsaw University of Technology

MSc program in Data Science has been developed
as a part of task 10 of the project
„NERW PW. Science - Education - Development - Cooperation”
co-funded by European Union from European Social Fund.